

Autonomous Trading Operating System

TSD Engineering Plan and Foundation Decision

Status	Proposed implementation baseline
Version	0.1
Date	2026-09-22
Repository	Multiagent_TSD-250926
Safety status	Live trading disabled until gates pass

This document is a design and delivery plan. Existing broker adapters and UI are prototypes; no live trading is authorized by this document.

Table of contents

Table of contents	2
TSD Engineering Plan and Foundation Decision	4
1. Executive decision	4
Recommended ownership boundary.....	4
Engine recommendation.....	4
2. The core semantic decision: target positions, not normal orders	5
2.1 Normal strategy output	5
2.2 Canonical TSD target-position contract	5
2.3 Examples.....	5
BTC quantity target.....	5
AAPL target weight.....	5
Samsung target notional	6
2.4 Delta calculation.....	6
2.5 Special direct-order strategies	6
2.6 Why this is the correct model	6
3. DaxAlgo evidence and reuse boundary.....	7
4. Repository current state	7
4.1 Current components	7
4.2 Blockers before live use.....	8
5. Target architecture.....	8
5.1 Service boundaries.....	8
6. Engine adoption decision process.....	9
6.1 NautilusTrader evaluation	9
6.2 Hummingbot API evaluation	9
6.3 No feature-list adoption	9
6.4 State ownership decision.....	9
6.5 Five concrete first deliverables	10
6.6 Persistence remains an integration responsibility	10
7. Requirement-to-upstream mapping	10
8. Specific next actions.....	11
Before any implementation commit.....	11
Engine spike.....	11
TSD bridge and existing UI	11
After the first demonstration.....	12
9. Canonical domain contracts	12
7.1 Core objects	12
7.2 Event envelope	12
7.3 API surface.....	12
10. Strategy lifecycle and agent boundary	13
Strategy API	13
Agent roles.....	13
11. Delivery roadmap.....	14
Phase 0 — Containment and baseline	14

Phase 1 — Engine compatibility spike and ADRs	14
Phase 2 — Canonical contracts and TSD bridge	14
Phase 3 — Safety facade and paper vertical slice	14
Phase 4 — DSM and real market-data integration	15
Phase 5 — Strategy builder, backtest, and promotion	15
Phase 6 — DaxAlgo-inspired workbench	15
Phase 7 — Bounded autonomy	15
Phase 8 — External paper and live certification	15
12. Testing strategy.....	15
Domain and bridge tests.....	15
Engine compatibility tests.....	16
Frontend tests.....	16
Security/operations.....	16
13. Persistence and observability.....	16
14. Security and live-trading gates	16
15. Risks and open decisions	17
16. DOCX generation and validation	17
17. Final recommendation.....	17
References.....	18
Local source evidence	18
External sources to verify against the pinned version	18
Appendix A — Architecture diagram source snapshots.....	18
Agent Proposal Sequence	19
Current State Repository	19
Engine Evaluation	19
Order State Machine	20
Strategy Promotion State Machine	20
Target Architecture.....	21
Target To Order Sequence	21

Autonomous Trading Operating System

TSD Engineering Plan and Foundation Decision

Document status: Proposed implementation baseline

Version: 0.1

Date: 2026-09-22

Scope: Multiagent_TSD-250926

Primary decision: Reuse a mature event-driven trading engine for OMS/EMS, paper execution, portfolio accounting, and broker lifecycle; make TSD the strategy-authoring, agent-orchestration, deployment, and operator-experience layer.

Safety warning: The current repository is a prototype. Existing broker adapters and trading UI are not production-ready and must not be treated as authorization to place live orders. Live trading remains disabled until the gates in this document are satisfied.

1. Executive decision

TSD should become an **agent-native trading operating system** supporting:

Market data

- > strategy creation
- > research and backtest
- > paper deployment
- > target-position output
- > risk approval
- > OMS/EMS order generation
- > paper or live execution
- > portfolio management
- > monitoring, reconciliation, and audit

The fundamental architectural correction is to avoid rebuilding an OMS, EMS, paper matching engine, position ledger, and broker lifecycle inside TSD if a mature engine already provides those capabilities.

Recommended ownership boundary

- **TSD owns:** strategy builder, strategy registry, agent workflows, backtest/deployment jobs, permissions, approval workflow, React workbench, and the bridge/API around the trading engine.
- **Adopted trading engine owns:** market-data adapters where supported, event-driven strategy runtime, order factory, execution engine, paper/sandbox execution, order/fill lifecycle, positions, portfolio P&L, risk primitives, cache, and reconciliation capabilities where supported.
- **TSD adds only:** policies or portfolio allocation rules genuinely missing from the selected engine, with tests proving the gap.

Engine recommendation

Run a short compatibility spike against **NautilusTrader** as the default candidate because TSD is intended to support crypto plus broader/multi-asset workflows including Interactive Brokers. Evaluate **Hummingbot API** as the alternative if the product becomes primarily crypto and the priority is a more application-level backend with ready-made trading, portfolio, market-data, backtesting, and orchestration routes.

Do not make the final engine decision from feature lists alone. Pin exact versions, build a paper vertical slice, test target-position semantics, verify supported venues and persistence, and measure operational fit. The chosen engine becomes the execution authority; TSD must not maintain a second parallel OMS path.

2. The core semantic decision: target positions, not normal orders

2.1 Normal strategy output

For ordinary alpha/strategy logic, **the strategy expresses the desired target position**. It does not create a broker order.

```
Strategy signal
-> TargetPositionIntent
-> portfolio/allocator calculates delta
-> risk engine approves or rejects
-> OMS creates an execution plan
-> EMS chooses order type, slicing, timing, and route
-> broker adapter submits orders
```

This is the same essential design used by DaxAlgo's sandbox model: a strategy writes to its own virtual book through `context.Book.SetTargetPosition(...)`; it cannot call `PlaceOrder`, does not see broker mode, and does not own order IDs or fills. The host decides separately whether the target is mirrored outward.

2.2 Canonical TSD target-position contract

```
{
  "intent_type": "target_position",
  "strategy_id": "btc-breakout",
  "strategy_version": "1.4.0",
  "account_scope": "paper-account-01",
  "instrument_id": "BTCUSDT",
  "target_type": "quantity",
  "target_value": "0.20",
  "currency": "BTC",
  "effective_at": "2026-09-22T10:15:30Z",
  "valid_until": "2026-09-22T10:16:00Z",
  "protective_stop_price": null,
  "reason": "Breakout confirmation",
  "signal_id": "uuid",
  "policy_version": "risk-policy-3",
  "correlation_id": "uuid"
}
```

Supported target types should be explicit:

- **quantity**: signed units; positive long, negative short, zero flat.
- **weight**: target fraction of portfolio equity, such as `0.03` for 3%.
- **notional**: target monetary value, with an explicit currency, such as `20,000,000 KRW`.

Do not overload one numeric field with ambiguous units. `target_quantity = 0.20`, `target_weight = 3%`, and `target_notional = 20,000,000 KRW` are different contracts.

2.3 Examples

BTC quantity target

```
Current BTCUSDT position: 0.18 BTC
Strategy target:           0.20 BTC
Delta:                     +0.02 BTC
OMS/EMS output:            BUY 0.02 BTC, subject to lot size, fees, risk, and execution policy
```

If the execution policy permits immediate market execution, the resulting order may be:

```
BUY 0.02 BTC MARKET
```

That order is **derived by the OMS/EMS**, not emitted by the strategy.

AAPL target weight

```
Portfolio equity:    $100,000
Target weight:       3%
```

Target notional: \$3,000
 Current AAPL value: \$1,500
 Required delta: +\$1,500
 OMS: Converts notional delta into shares using price, FX, lot size, fractional-share capability, and risk policy.

Samsung target notional

Target notional: 20,000,000 KRW
 OMS: Converts KRW value into shares using the current trusted price, account currency, FX policy, venue lot size, and available cash.

The result is not guaranteed to be exactly 20,000,000 KRW because of price movement, tick/lot rounding, fees, partial fills, liquidity, FX, and risk limits. The execution record must show requested target, calculated delta, accepted order quantity, residual, and final position.

2.4 Delta calculation

The OMS must calculate against a clearly defined position snapshot:

```

projected_position = settled_position
                   + filled_quantity
                   + approved_open_order_effect
                   - reserved/cancelled quantities
  
```

```

delta = target_position - projected_position
  
```

The exact formula depends on the adopted engine's reservation and order-state model. The implementation must document whether open orders are included, how partial fills are treated, and how concurrent target updates are versioned.

The OMS then applies:

1. Instrument precision and lot-size rules.
2. Minimum notional and price-grid rules.
3. Account cash/margin and reservation rules.
4. Portfolio exposure and concentration rules.
5. Existing open-order netting or cancellation policy.
6. Execution urgency and order-type policy.
7. Broker/venue capability and route selection.

2.5 Special direct-order strategies

Direct order generation is allowed only for special **execution strategies**, not normal alpha:

- VWAP/TWAP slicers.
- Iceberg/hidden-liquidity execution.
- Market-making quote placement.
- Hedging or spread legs requiring synchronized orders.
- Latency-sensitive microstructure strategies.
- Cancel/replace controllers.

Even these strategies must not call a broker SDK directly. They emit a typed `ExecutionIntent` or `OrderInstruction` into the same risk and execution boundary. The engine must mark the intent as `direct_execution` and apply stricter permissions, rate limits, and audit requirements.

```

Normal alpha strategy  -> TargetPositionIntent
Execution strategy     -> ExecutionIntent / OrderInstruction
Both                  -> Risk -> adopted OMS/EMS -> broker adapter
Neither               -> raw broker SDK access
  
```

2.6 Why this is the correct model

Target positions make backtest, paper, and live strategy code share the same semantic path. They reduce duplicate order logic, make reversals declarative, improve explainability, and prevent strategies from depending on broker-specific order IDs. The execution engine remains responsible for the real-world problems that strategies should not solve: retries, partial fills, cancellation races, routing, reconciliation, and broker-specific constraints.

3. DaxAlgo evidence and reuse boundary

The local DaxAlgo repository confirms the target-position design:

- `DaxAlgo-Terminal/sdk/ai-context/daxalgo-strategy-context.md` states that a strategy's only output is its own virtual book and that `context.Book.SetTargetPosition(...)` declares the desired position.
- `DaxAlgo-Terminal/sdk/ai-context/skills/risk-and-exits.md` states that the host works out the difference from the current position and that a reversal is one new target rather than a close-plus-open order sequence.
- `DaxAlgo-Terminal/src/windows/Execution/TradingTerminal.Execution/TradeIntent.cs` explicitly distinguishes `TargetPosition` from `Delta` quantity modes.
- `DaxAlgo-Terminal/src/windows/Execution/TradingTerminal.Execution/RiskEngine.cs` projects `after` and `order` quantities differently for target versus delta intents.
- `DaxAlgo-Terminal/src/windows/Execution/TradingTerminal.Execution/Oms/OrderDomain.cs` verifies that the signed order quantity equals target minus current position for target-position instructions.

The useful TSD reuse is therefore the **semantic contract and UX pattern**, not WPF controls:

DaxAlgo concept	TSD implementation
Virtual book	Engine strategy position/portfolio scope
<code>`SetTargetPosition`</code>	<code>`TargetPositionIntent`</code> adapter
Target vs delta	Versioned quantity-mode contract
Host-controlled Paper/Real	Server-authoritative deployment/account capability
Activity log	Event/audit stream
Strategy catalog	React strategy registry and deployment cards
Execution console	Engine events projected into TSD APIs

4. Repository current state

4.1 Current components

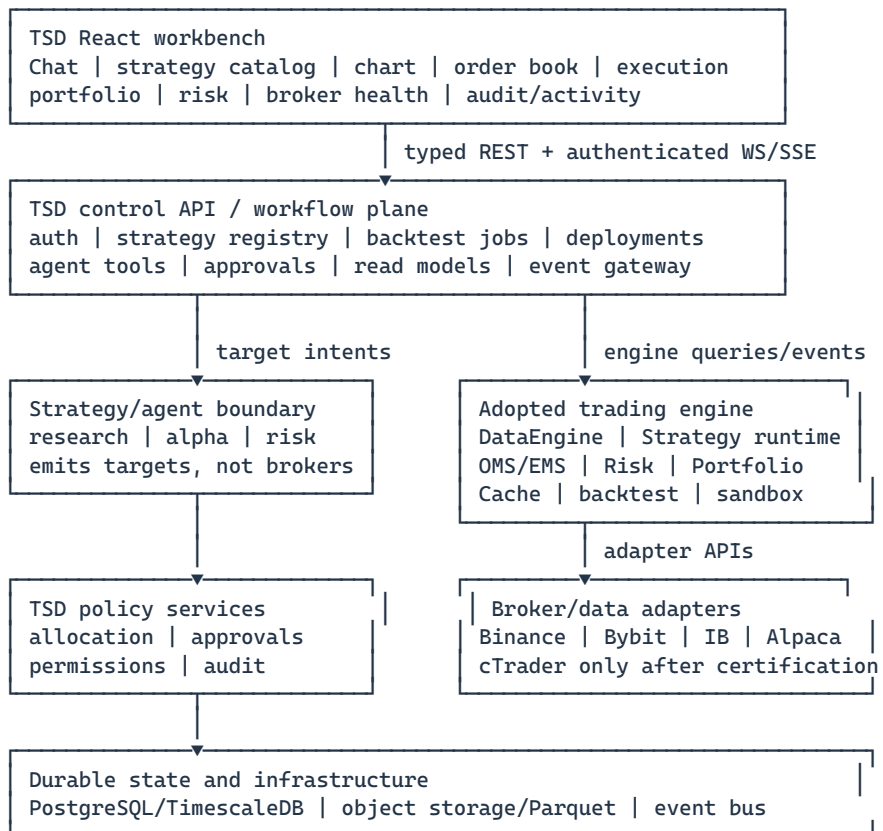
Area	Current state	Decision
React/Vite UI	CAE workflow shell plus uncommitted <code>`/trading`</code> prototype	Reuse shell/query patterns; replace synthetic trading state
FastAPI	CAE APIs plus uncommitted broker routes	Keep as TSD control API; remove direct route-to-broker ownership
<code>`tsd/dsm`</code>	Coinbase WebSocket/ZMQ order-book prototypes	Reuse parsing concepts only after feed contract/recovery tests
<code>`tsd/osm`</code>	Legacy exchange managers with config/model inconsistencies	Reference/migrate selectively; do not keep as second OMS
<code>`tsd/tsm`</code>	CSP training examples	Keep as learning material; build TSD engine adapter/runtime
DaxAlgo	Mature local target-position and execution semantics	Reuse conceptual model and evidence, not WPF controls
NautilusTrader candidate	External mature engine candidate	Compatibility spike; likely default for multi-

Area	Current state	Decision
		asset direction
Hummingbot candidate	External crypto-oriented application/backend candidate	Alternative if crypto-only priority wins

4.2 Blockers before live use

- Apparent exchange credentials are present in repository files/history; revoke and rotate them.
- Authentication is a constant-token stub and trading routes do not enforce meaningful authorization.
- CORS is wildcard and the current WebSocket is not an authenticated event stream.
- The frontend Paper/Live control is cosmetic and not a safety boundary.
- The current broker pool is process-local and order results are unstructured dictionaries.
- No durable ledger, order event store, idempotency, reconciliation, or kill-switch service exists.
- The current cTrader path is incomplete.
- Legacy OSM and current backend broker adapters are duplicate execution paths.
- No strategy deployment runtime or backtest service is connected to the agent layer.
- No backend/frontend trading test suite or CI gate is established.

5. Target architecture



5.1 Service boundaries

8. **Control API:** auth, catalog, strategy/deployment commands, read models, approval workflows, browser gateway. No raw broker SDK calls.
9. **Trading-engine runtime:** adopted engine process or service owning order/fill/position/portfolio lifecycle.
10. **Market-data service:** configured engine adapters or TSD feed adapter; sequence health and recording remain explicit.

11. **Strategy runner:** starts approved versions and emits target/execution intents.
12. **Policy/audit service:** permissions, risk overlays missing from engine, approvals, immutable decision trail.
13. **Persistence:** engine persistence plus TSD registry/audit/projected read models; one declared source of truth per aggregate.

A single development deployment may initially host these modules together. Live certification must isolate the execution process from the CAE application.

6. Engine adoption decision process

6.1 NautilusTrader evaluation

Evaluate NautilusTrader first for the multi-asset direction. Verify, at the pinned version:

- Strategy lifecycle and target-position or equivalent position-management semantics.
- Order factory, execution engine, order/fill events, cache, positions, portfolio P&L, and risk engine.
- Backtest and sandbox execution with deterministic configuration.
- Supported Binance, Bybit, Interactive Brokers, and required market-data paths.
- Persistence, restart recovery, reconciliation, and external adapter behavior.
- Python/Rust deployment model and compatibility with FastAPI workers.
- Whether TSD must add target-to-delta allocation or whether the engine has a suitable position-management command.

6.2 Hummingbot API evaluation

Evaluate Hummingbot API if crypto is the dominant scope. Verify:

- FastAPI/PostgreSQL application integration.
- Trading, portfolio, market-data, backtesting, and orchestration routes.
- How spot balances and perpetual positions are represented.
- Strategy lifecycle, bot deployment, and event streaming.
- Whether its application-level abstractions can accept TSD target-position intents without forcing TSD into a direct-order model.
- Venue coverage and suitability for IB/non-crypto expansion.

6.3 No feature-list adoption

The compatibility spike must produce a decision matrix and a running demo. The engine is accepted only if it passes:

```
market data -> target position -> OMS/EMS delta -> paper order -> fill
              -> position/P&L -> restart -> event/UI projection
```

The demo must prove duplicate target updates, partial fills, reversal, cancellation, stale data, risk rejection, and strategy attribution. If neither candidate meets the target-position contract cleanly, build a thin TSD adapter around the better engine rather than rebuilding the engine.

6.4 State ownership decision

The selected engine and TSD must not become two competing trading systems. Record this ownership table in the first ADR:

State	Authoritative owner
Users, permissions, proposals, approvals	TSD application database
Strategy versions and deployment configuration	TSD application database
Internal order lifecycle, fills, and positions	Adopted engine runtime and its configured recovery mechanisms
Trading P&L and account calculations	Adopted engine accounting/portfolio
Actual external orders, executions, and balances	Broker/venue evidence, reconciled into the engine

State	Authoritative owner
Browser tables and charts	Read-only projections of engine/TSD events
Research evidence and agent explanations	TSD research/agent records

A separate double-entry reporting ledger may be added later for formal accounting, transfers, or customer reporting. It must have an explicit purpose and reconciliation rules; it must not silently become a second execution-state authority. Engine accounting and P&L must not be described as satisfying every possible accounting requirement.

6.5 Five concrete first deliverables

Deliverable	Developer produces	Completion evidence
1. Contained baseline	Preserved working tree, credential remediation, real authentication, restricted trading routes	Existing prototype remains recoverable; unauthorized requests cannot trade
2. Pinned engine integration	Exact engine version, matching examples, one strategy, one instrument, one data source	Strategy runs in backtest and local paper using the selected engine
3. TSD trading bridge	Command intake, idempotency records, runtime launcher, engine-event adapter	Authenticated request produces an engine order and traceable outcome
4. Existing UI connected	Real orders, fills, positions, P&L, and runtime/feed status in the React prototype	Displayed records correspond to engine IDs/events; synthetic data is clearly separated
5. Recovery and agent demonstration	Restart/retry tests plus scoped `propose_target` or `propose_order` tool	Approved proposal executes in paper and traces through its resulting position

Begin with one spot instrument, one simulated account, and no leverage or short selling. Choose a venue already covered by the adopted engine; Bybit is a candidate for the crypto direction, not a claim that the current connection works.

6.6 Persistence remains an integration responsibility

Adopting an engine does not remove recovery work. The bridge must durably associate:

TSD command ID <-> deployment ID <-> engine client order ID <-> broker order ID

If the worker restarts after submission, it must resolve that existing identity before deciding whether another submission is necessary. A PostgreSQL command table is sufficient for the first single-worker implementation. A durable event bus is optional until scale or isolation requires it.

Test the adopted simulator's recoverable state explicitly. Restoring local paper orders and simulated balances is different from reconciling with a remote broker. Determinism means comparing normalized trading outputs under fixed data/configuration; hashing wall-clock timestamps or random IDs is not evidence of deterministic behavior.

7. Requirement-to-upstream mapping

The implementation plan is not a list of capabilities to rebuild. It is a mapping from requirement to upstream capability, TSD integration work, and acceptance evidence.

Requirement	Existing upstream implementation	TSD integration work	Acceptance test
Strategy target position	DaxAlgo virtual-book semantics; selected engine position/order APIs	Adapt `TargetPositionIntent`, including quantity/weight/notional normalization	BTC target 0.20 with current 0.18 yields a derived +0.02 plan
Order state and fills	Selected engine execution lifecycle	Map engine events to versioned	Partial fill, cancel race, reject, fill

Requirement	Existing upstream implementation	TSD integration work	Acceptance test
		TSD DTOs/read models	are visible with engine IDs
Paper matching	Selected engine simulator/sandbox	Configure fee, latency, liquidity, and starting balances	Fixed fixture produces repeatable normalized fills
Positions and P&L	Selected engine cache/accounting/portfolio	Query, filter, attribute, and project to React	UI matches engine position/P&L values after fills
Broker integration	Maintained engine adapter where available	Configure credentials/capabilities; add adapter only for proven gap	Sandbox/paper lifecycle contract passes without raw TSD broker calls
Backtesting	Selected engine backtest interfaces	Submit jobs, store manifests/results, render metrics	Same strategy version runs in backtest and paper
Reconciliation/recovery	Engine persistence/reconciliation capabilities plus broker evidence	Configure supported recovery; persist TSD command-to-engine identity	Restart after submit does not duplicate order
Agent approval	Not an engine responsibility	Proposal, authority, approval, audit, and final target command	Agent proposal cannot trade until approved and rechecked
UI workbench	Not an engine responsibility	REST/WebSocket bridge and DaxAlgo-inspired screens	React records correspond to engine events, not mock values

8. Specific next actions

Before any implementation commit

14. Run `git status --short --branch` and save the diff/untracked inventory.
15. Inspect every changed/untracked trading file for secrets before staging anything.
16. Revoke/rotate any exposed credentials; do not create a WIP commit until this is complete.
17. Replace the constant-token auth and wildcard CORS, or disable trading mutations until proper auth exists.
18. Add a `.gitignore`/secret-scanning check for environment files, keys, and broker credentials.
19. Mark the existing trading additions as prototype/WIP and keep them recoverable separately.

Engine spike

20. Choose an exact NautilusTrader release and matching documentation/API examples; record the pin in an ADR.
21. Install it in an isolated environment and run its official minimal strategy/backtest example unchanged.
22. Run its local paper/sandbox example with one spot instrument and one simulated account.
23. Implement one TSD adapter that turns `TargetPositionIntent` into the engine's supported position/order command.
24. Demonstrate `BTCUSDT target_quantity = 0.20` with a current position of `0.18`; record the derived order, fill, position, and P&L.
25. Repeat the demonstration with one supported venue/instrument only; do not mix BTC, AAPL, and Samsung until asset/FX semantics are proven.
26. Run the same strategy in backtest and paper; record version, config, data, engine, and execution-model identifiers.
27. Test partial fill, reversal, duplicate target update, cancel race, stale data, and risk rejection using the adopted engine's configured behavior.

TSD bridge and existing UI

28. Add a TSD command ID -> deployment ID -> engine client order ID persistence record in PostgreSQL.
29. Add an engine-event adapter for orders, fills, positions, P&L, runtime status, and feed status.
30. Connect the current React /trading prototype to those read models and clearly label any remaining synthetic fixtures.

31. Replace the frontend Paper/Live toggle as an authority; show mode/capabilities from the server.
32. Add a scoped `propose_target` agent tool that produces a proposal, not an order.
33. Require operator approval, server-side authority/policy validation, engine submission, and a traceable UI timeline.
34. Restart the bridge after an engine submission and prove it resolves the existing identity instead of duplicating the order.

After the first demonstration

35. Decide whether Hummingbot remains a serious alternative based on the same target-position and recovery test, not its route count.
36. Add more instruments/venues only after the first slice is stable.
37. Add weight/notional targets only after valuation, FX, lot-size, and rounding rules are explicit.
38. Add direct `ExecutionIntent` only for specialized execution strategies and keep it behind stricter permissions.
39. Keep the legacy `tsd/osm` path disabled until a demonstrated capability gap justifies migration.
40. Start live certification only after paper soak, reconciliation, kill-switch, and governance approvals.

9. Canonical domain contracts

7.1 Core objects

- **Instrument**: canonical symbol, asset class, venue mappings, tick/lot/settlement/currency.
- **Account**: broker/account identifier, environment, base currency, enabled capabilities.
- **StrategyVersion**: immutable artifact/config hash and required data capabilities.
- **TargetPositionIntent**: quantity/weight/notional target plus provenance and validity.
- **ExecutionIntent**: special direct execution instruction, still subject to risk and engine.
- **RiskDecision**: accepted/rejected/modified, policy version, checks and explanations.
- **OrderPlan**: OMS-produced desired order delta, order type, route, residual, and expiry.
- **BrokerOrder**: external order mapping and client ID.
- **Fill**: immutable execution economics and provider ID.
- **Position**: engine/projected position with strategy/account attribution.
- **PortfolioSnapshot**: equity, cash, exposure, P&L, reservations.
- **Deployment**: strategy version, account, mode, permissions, engine/runtime version.
- **AuditEvent**: actor, command, decision, approval, outcome, correlation.

7.2 Event envelope

```
{
  "event_id": "uuid",
  "event_type": "trading.position.updated.v1",
  "schema_version": 1,
  "aggregate_type": "position",
  "aggregate_id": "uuid",
  "aggregate_version": 12,
  "occurred_at": "2026-09-22T10:15:30.123Z",
  "correlation_id": "uuid",
  "causation_id": "uuid",
  "producer": "engine-bridge",
  "payload": {}
}
```

Use at-least-once delivery and idempotent consumers. Never claim exactly-once semantics without proving the complete path.

7.3 API surface

Endpoint	Responsibility
<code>POST /api/v1/trading/targets/preview`</code>	Convert target to projected delta/order plan and show risk result

Endpoint	Responsibility
`POST /api/v1/trading/targets`	Submit idempotent target-position command
`POST /api/v1/trading/execution-intents`	Restricted special execution path
`GET /api/v1/trading/orders`	Engine order projection
`GET /api/v1/trading/fills`	Fill history
`GET /api/v1/trading/positions`	Engine/projected positions
`GET /api/v1/trading/portfolio`	Equity, cash, exposure, realized/unrealized P&L
`GET /api/v1/trading/brokers`	Data/execution health and capabilities
`POST /api/v1/trading/deployments`	Create approved paper/live deployment
`POST /api/v1/trading/deployments/{id}/pause`	Pause strategy/deployment
`POST /api/v1/trading/controls/kill-switch`	Privileged halt
`GET /api/v1/trading/events/stream`	Authenticated resumable event stream

All mutations require principal authorization, a correlation ID, and idempotency where retries could duplicate intent.

10. Strategy lifecycle and agent boundary

Draft

- > Validated
- > Backtested
- > Paper-deployed
- > Paper-soak-approved
- > Live-approved
- > Live-canary
- > Live
- > Paused
- > Archived

Strategy API

A strategy receives market data, clock, portfolio context, and execution feedback. It emits:

- Signals for explainability.
- `TargetPositionIntent` for normal alpha.
- `ExecutionIntent` only for explicitly classified execution strategies.
- Health/heartbeat events.

It cannot receive broker credentials or direct SDK objects.

Agent roles

Agent	Allowed initially	Not allowed initially
Research	Read data, summarize evidence	Place orders
Strategy builder	Create versioned strategy artifacts	Enable live mode

Agent	Allowed initially	Not allowed initially
Backtest	Run deterministic jobs	Promote itself
Risk	Evaluate policies and explain	Increase limits
Portfolio	Read/recommend allocations	Bypass risk
Execution	Observe engine lifecycle	Raw broker access
Supervisor	Pause/kill deployments	Disable kill switch
Human approver	Approve configured promotions	Override hard safety controls

Agent tools begin read-only and proposal-based: `get_market_snapshot`, `get_portfolio`, `request_backtest`, `get_risk_summary`, `propose_target_position`, and `explain_decision`. A proposal is not an order. The server re-runs risk when a proposal becomes an approved target command.

11. Delivery roadmap

Phase 0 — Containment and baseline

- Rotate/revoke exposed secrets.
- Preserve/classify the uncommitted trading prototype.
- Replace constant-token auth and wildcard CORS.
- Disable live mutations by default.
- Select package managers and repair lock drift.
- Repair README conflict markers and add CI/secret scanning.

Exit criteria: anonymous calls cannot mutate trading state; secret scan passes; prototype is preserved separately; clean install and lint/typecheck path is documented.

Phase 1 — Engine compatibility spike and ADRs

- Pin NautilusTrader candidate version.
- Evaluate Hummingbot API in parallel at the API-contract level.
- Implement a thin TSD target-position adapter.
- Demonstrate BTCUSDT quantity target, AAPL weight target, and KRW notional target in paper mode where supported.
- Test partial fill, reversal, restart, and strategy attribution.
- Decide engine, storage, bus, and deployment boundary.

Exit criteria: one engine is selected with evidence; target-position semantics are explicit; no custom OMS rebuild is scheduled unless a tested gap remains.

Phase 2 — Canonical contracts and TSD bridge

- Define versioned Pydantic/domain schemas and generate TypeScript types.
- Implement target/weight/notional normalization and conversion.
- Add engine event bridge to REST/WebSocket read models.
- Add strategy registry/version/deployment records.

Exit criteria: UI can display engine orders, fills, positions, portfolio, and target provenance from one paper deployment.

Phase 3 — Safety facade and paper vertical slice

- Add server-side permissions, idempotency, risk overlays, approvals, and kill switches.
- Route normal target intents through engine OMS/EMS.
- Route special execution intents through a restricted engine adapter.
- Add deterministic replay and paper-simulation acceptance tests.

Exit criteria: target -> derived order -> fill -> position/P&L is durable/repeatable and safe under retries, stale data, partial fills, and cancellation.

Phase 4 — DSM and real market-data integration

- Adopt engine feed adapters where sufficient.
- Keep TSD DSM only for missing feeds, normalization, recording, or browser projection.
- Add snapshot/delta sequence health, stale/invalid status, recording/replay, and authenticated browser stream.

Exit criteria: no synthetic trading data in the workbench; feed gaps invalidate the book and recover visibly.

Phase 5 — Strategy builder, backtest, and promotion

- Translate user rules/workflows into immutable strategy versions.
- Submit backtest jobs through the selected engine.
- Add no-look-ahead, walk-forward, dataset/config/code hashes, and paper soak.
- Add sandbox/resource limits and heartbeats.

Exit criteria: the same strategy version runs in backtest and paper with reproducible intent/order attribution.

Phase 6 — DaxAlgo-inspired workbench

- Broker/data status panel.
- Execution console.
- Strategy catalog.
- Real chart/order book/tape.
- Portfolio and risk panels.
- Deployment timeline and audit log.
- Server-authoritative Paper/Live state.

Exit criteria: all key states—accepted, rejected, partial, filled, canceled, stale feed, disconnected, paused, and kill switch—are visible and tested.

Phase 7 — Bounded autonomy

- Add read/proposal agent tools.
- Add human approval thresholds and per-agent budgets.
- Correlate prompt, evidence, proposal, approval, risk decision, and engine order.

Exit criteria: agents cannot bypass target-position, risk, engine, or audit boundaries.

Phase 8 — External paper and live certification

- Certify one venue at a time.
- Alpaca/IB/cTrader are not assumed to be supplied by the adopted engine; adapter support must be verified and tested.
- Keep cTrader disabled until complete protocol/lifecycle tests pass.
- Use paper soak, reconciliation drills, kill-switch drills, two-person approval, and low-limit canary.

Exit criteria: documented security/correctness/operations gate signed by engineering, security, operations, and product owners.

12. Testing strategy

Domain and bridge tests

- Target quantity/weight/notional conversion with Decimal-safe arithmetic.
- Target-to-delta calculations against settled, filled, and open-order states.
- Lot-size, tick-size, FX, min-notional, and residual handling.
- Order/position event ordering and deduplication.
- Reversal and partial-fill semantics.
- Risk overlays and kill switches.

- Strategy attribution and correlation IDs.
- Engine adapter contract tests using fake/sandbox providers.

Engine compatibility tests

- Pin exact package version and matching docs/examples.
- Verify strategy callbacks, order factory, `submit_order`, fill events, position events, cache queries, portfolio P&L, and reconciliation APIs.
- Verify persistence/restart behavior rather than relying on in-memory cache.
- Exercise target updates faster than fills and define coalescing/version rules.

Frontend tests

- Typecheck/build/lint.
- React Testing Library and MSW contract tests.
- Playwright flows for target preview, approval, execution timeline, cancellation, pause, kill switch, reconnect, and stale feed.
- Prove that a frontend toggle cannot enable live trading without server capability.

Security/operations

- Auth matrix and WebSocket authorization.
- Agent tool scope/budget/expiry tests.
- Secret scan and dependency scan.
- Fault injection for engine restart, broker timeout, duplicate event, sequence gap, stale data, and reconciliation break.
- No real order-submission tests in normal CI.

13. Persistence and observability

Use the adopted engine's supported persistence for trading lifecycle and PostgreSQL/TimescaleDB for TSD-owned registry, approvals, audits, and read models, provided ownership is explicit. Do not duplicate authoritative order/position ledgers accidentally.

TSD-owned records should include:

- Principals, roles, and permissions.
- Strategy versions, artifacts, parameters, dataset manifests.
- Deployments and approvals.
- Target intents and execution-intent provenance.
- Risk decisions and policy versions.
- Engine IDs and external broker IDs.
- Audit events and correlation chains.

Minimum metrics:

- Target-to-plan, plan-to-ack, and ack-to-fill latency.
- Risk rejections by reason.
- Target convergence residual.
- Engine/broker reconnects and reconciliation breaks.
- Feed age, sequence gaps, and recovery duration.
- Strategy heartbeat and event lag.
- Position/P&L drift between engine and broker.
- Agent proposals, approvals, expirations, and rejects.

14. Security and live-trading gates

Live must be impossible by default and require all of:

- Maintained identity provider, MFA, roles, account scoping, and short-lived service identities.
- Secret manager and credential rotation.
- Authenticated WebSocket/SSE with origin and subscription checks.

- Server-side mode/capability checks; no cosmetic UI authority.
- Target/order allowlists, size/notional limits, price collars, stale-data checks, and kill switches.
- Idempotency, immutable audit, engine/broker reconciliation, and restart recovery.
- Paper soak and canary evidence.
- Tested cancel-all/flatten and incident runbooks.
- Two-person promotion approval.

A typed confirmation is a useful UX gate but is not a substitute for server-side policy.

15. Risks and open decisions

Risk/open decision	Recommended response
Engine version/API drift	Pin exact version and run compatibility tests in CI
Nautilus versus Hummingbot	Decide from target-position compatibility, venue needs, persistence, and operations—not feature count
Current engine target semantics	Build a thin adapter if engine accepts delta/orders; do not force strategies to emit broker orders
Crypto versus multi-asset MVP	Use one aligned asset/venue for the paper slice; do not mix unrelated data and execution
Cross-strategy netting	Define whether engine accounts are per-strategy, per-portfolio, or centrally allocated
Weight/notional conversion	Define FX source, valuation timestamp, rounding, and residual policy
Direct execution strategies	Separate 'ExecutionIntent', require stricter permissions and rate limits
Legacy OSM duplication	Deprecate once selected engine adapter passes certification
cTrader support	Treat as unavailable until protocol and lifecycle tests pass
Agent autonomy	Start proposal-only; require human/policy approval for live promotions
Market-data licensing	Resolve before recording or redistributing data

16. DOCX generation and validation

The DOCX is generated from this Markdown plus metadata and diagram source files; it is not the only authoritative copy.

Planned files:

- docs/trading-os-engineering-plan.md
- docs/trading-os-engineering-plan.yaml
- tools/generate_trading_plan.py
- docs/diagrams/*.mmd
- docs/generated/Autonomous_Trading_OS_Engineering_Plan.docx

The generator must create title, heading, table, callout, code, header/footer, page number, table-of-contents field, landscape sections for wide tables, and embedded diagrams. Validation must reopen the DOCX, check headings/tables/acceptance criteria/risk register, reject placeholders/secrets/merge markers, validate the Open XML package, and use Word on Windows to update the TOC and export a PDF for visual inspection.

17. Final recommendation

The TSD foundation should be:

```
DaxAlgo semantic model
+ mature adopted trading engine
+ TSD strategy/agent/workflow layer
+ TSD React workbench
+ explicit target-position -> OMS/EMS bridge
```

Do **not** begin by building four trading screens, more broker connectors, or a custom OMS. First prove one target-position strategy through the selected engine in paper mode:

```
BTCUSDT target quantity 0.20
-> engine/OMS calculates delta
-> EMS creates BUY 0.02 BTC MARKET when current position is 0.18
-> paper fill
-> position/P&L update
-> event reaches TSD UI
```

Then prove the same path for an AAPL weight target and a Samsung KRW-notional target where the selected engine and venue support the required instrument/account semantics. Only after this vertical slice is correct should TSD add autonomous agents and live execution.

References

Local source evidence

- DaxAlgo-Terminal/sdk/ai-context/daxalgo-strategy-context.md
- DaxAlgo-Terminal/sdk/ai-context/skills/risk-and-exits.md
- DaxAlgo-Terminal/src/windows/Execution/TradingTerminal.Execution/TradeIntent.cs
- DaxAlgo-Terminal/src/windows/Execution/TradingTerminal.Execution/RiskEngine.cs
- DaxAlgo-Terminal/src/windows/Execution/TradingTerminal.Execution/Oms/OrderDomain.cs
- Multiagent_TSD-250926/backend/back-end/trading/api.py
- Multiagent_TSD-250926/tsd/dsm/publisher/order_book_publisher.py
- Multiagent_TSD-250926/tsd/osm/order_service_manager.py
- Multiagent_TSD-250926/tsd/tsm/README.md

External sources to verify against the pinned version

- NautilusTrader architecture: <https://nautilustrader.io/docs/latest/concepts/architecture/>
- NautilusTrader strategies: <https://nautilustrader.io/docs/latest/concepts/strategies/>
- NautilusTrader repository/version notes: https://github.com/nautechsystems/nautilus_trader
- NautilusTrader integrations: <https://nautilustrader.io/docs/latest/integrations/>
- Hummingbot API: <https://hummingbot.org/hummingbot-api/>
- Hummingbot API routers: <https://hummingbot.org/hummingbot-api/routers/>
- Hummingbot API repository: <https://github.com/hummingbot/hummingbot-api>
- Hummingbot Dashboard: <https://github.com/hummingbot/dashboard>

External claims are implementation inputs, not acceptance evidence. The compatibility spike must verify the exact pinned release and supported adapters.

Appendix A — Architecture diagram source snapshots

The source-controlled Mermaid files are the authoritative diagram definitions. PNG snapshots are embedded here so the DOCX remains readable without a Mermaid renderer.

Agent Proposal Sequence

Agent Proposal Sequence

```
sequenceDiagram
    participant A as Agent
    participant T as TSD Tool Gateway
    participant H as Human/Policy Approval
    participant R as Risk Layer
    participant E as Engine OMS/EMS
    participant U as UI/Audit
    A->>T: propose_target_position
    T->>T: Validate scope, expiry, budget, evidence
    T-->>U: Proposal with rationale and correlation ID
    H->>T: Approve or reject proposal
    T->>R: Re-run risk on approved target
    R-->>T: Decision
    T->>E: Submit approved target only
    E-->>U: Derived orders/fills/positions
    T-->>U: Prompt, evidence, approval, decision, outcome audit chain
```

Figure A-1: agent-proposal-sequence.mmd

Current State Repository

Current State Repository

```
flowchart LR
    FE[React/Vite CAE + trading prototype]
    API[FastAPI CAE + direct broker routes]
    AG[CAE agents]
    DSM[tsd/dsm Coinbase WebSocket/ZMQ prototype]
    OSM[tsd/osm legacy exchange managers]
    TSM[tsd/tsm CSP training examples]
    BROKERS[(Broker/exchange APIs)]
    MEM[(Process-local state)]
    FE --> API
    API --> AG
    API --> BROKERS
    DSM --> OSM
    DSM --> MEM
    TSM -. training only .-> DSM
    OSM --> BROKERS
    FE -. synthetic trading data .-> FE
```

Figure A-2: current-state-repository.mmd

Engine Evaluation

Engine Evaluation

```
flowchart LR
    N[NautilusTrader\nprimary candidate]
    H[Hummingbot API\ncrypto-focused alternative]
    SPIKE[Version-pinned compatibility spike\nBTC target -> derived order -> fill -> P&L  
-> restart]
    DECIDE[Select one execution authority\nNo parallel OMS]
    TSD[TSD owns strategy builder\nagent workflows\ndeployments\nReact workbench]
    N --> SPIKE
    H --> SPIKE
    SPIKE --> DECIDE
    DECIDE --> TSD
```

Figure A-3: engine-evaluation.mmd

Order State Machine

Order State Machine

```
stateDiagram-v2
    [*] --> RECEIVED
    RECEIVED --> RISK_REJECTED
    RECEIVED --> APPROVED
    APPROVED --> SUBMISSION_PENDING
    APPROVED --> CANCELED
    SUBMISSION_PENDING --> ACKNOWLEDGED
    SUBMISSION_PENDING --> REJECTED
    ACKNOWLEDGED --> PARTIALLY_FILLED
    ACKNOWLEDGED --> FILLED
    ACKNOWLEDGED --> CANCEL_PENDING
    ACKNOWLEDGED --> EXPIRED
    PARTIALLY_FILLED --> PARTIALLY_FILLED
    PARTIALLY_FILLED --> FILLED
    PARTIALLY_FILLED --> CANCEL_PENDING
    CANCEL_PENDING --> CANCELED
    CANCEL_PENDING --> PARTIALLY_FILLED
    CANCEL_PENDING --> FILLED
    RISK_REJECTED --> [*]
    REJECTED --> [*]
    CANCELED --> [*]
    EXPIRED --> [*]
    FILLED --> [*]
```

Figure A-4: order-state-machine.mmd

Strategy Promotion State Machine

Strategy Promotion State Machine

```
stateDiagram-v2
    [*] --> Draft
    Draft --> Validated
    Validated --> Backtested
    Backtested --> PaperDeployed
    PaperDeployed --> PaperSoakApproved
    PaperSoakApproved --> LiveApproved
    LiveApproved --> LiveCanary
    LiveCanary --> Live
    Live --> Paused
    LiveCanary --> Paused
    PaperDeployed --> Paused
    Paused --> PaperDeployed
    Paused --> Archived
    Live --> Archived
```

Figure A-5: strategy-promotion-state-machine.mmd

Target Architecture

Target Architecture

```

flowchart TB
    UI[TSD React workbench\nchat/catalog/chart/book/execution/portfolio]
    CTRL[Authenticated TSD control API\nregistry/deployments/approvals/read models]
    AG[Research/strategy/risk/portfolio agents\nread + proposal tools]
    BRIDGE[Target-position and engine event bridge]
    ENGINE[Adopted trading engine\nstrategy runtime | OMS/EMS | risk | portfolio |
    backtest | sandbox]
    POLICY[TSD policy/audit layer\npermissions | allocation | approvals | kill switch]
    DATA[Engine data adapters + TSD DSM where needed]
    BROKER[Binance | Bybit | Interactive Brokers | Alpaca\nTrader only after
    certification]
    STORE[(PostgreSQL/TimescaleDB\nregistry/audit/read models)]
    ARCH[(Parquet/object storage\nmarket datasets/replay)]
    BUS[(Durable event bus)]

    UI --> CTRL
    CTRL --> BRIDGE
    CTRL --> AG
    AG --> POLICY
    POLICY --> BRIDGE
    BRIDGE --> ENGINE
    DATA --> ENGINE
    ENGINE --> BROKER
    ENGINE --> BUS
    ENGINE --> STORE
    CTRL --> STORE
    DATA --> ARCH
    BUS --> CTRL
  
```

Figure A-6: target-architecture.mmd

Target To Order Sequence

Target To Order Sequence

```

sequenceDiagram
    participant S as Strategy
    participant B as TSD Bridge
    participant P as Portfolio/Allocator
    participant R as Risk Policy
    participant O as Adopted OMS/EMS
    participant E as Execution Engine
    participant X as Broker/Paper Venue
    participant U as TSD UI

    S->>B: TargetPositionIntent(BTCUSD=0.20)
    B->>P: Read position/open orders/valuation
    P-->>B: Current 0.18, delta +0.02
    B->>R: Preview target and constraints
    R-->>B: Approved / modified / rejected
    B->>O: Approved target + policy version
    O->>E: Derived BUY 0.02 MARKET plan
    E->>X: Submit paper/live order
    X-->>E: Ack / partial fill / fill
    E-->>B: Order, fill, position, P&L events
    B-->>U: Correlated lifecycle timeline
  
```

Figure A-7: target-to-order-sequence.mmd